챕터 4. 사진을 글자로 바꿔서 보낸다. Postman ↔ Spring 이미지 업로드 (Base64)

이번 챕터가 끝나면

- 왜 사진을 곧바로 JSON에 담을 수 없는지 이해합니다 (바이너리 ↔ 텍스트의 경계)
- 사진을 긴 문자열로 바꿔서 서버에 올리고 다시 파일로 되돌리는 흐름을 손으로 만들어 봅니다 (Base64 인코딩·디코딩)
- 저장된 파일을 URL 한 줄로 브라우저에 띄웁니다 (정적 리소스 매핑)
- 이 방식의 한계를 숫자로 체감합니다 (용량 33퍼센트 증가·DB 비대·메모리 이슈)

이번 챕터는 "사진 한 장을 JSON 안에 욱여넣는 이야기"입니다

챕터 3에서 로그인·세션 문제를 끝냈습니다. 동료가 말미에 던진 한마디는 "프로필 사진 바꾸고 싶은데"였습니다. 사용자가 실제로 무엇인가를 **남기는** 첫 기능이 시작됩니다. 그런데 막상 Postman을 열고 보면 문제가 하나 있습니다. REST API는 JSON으로 말하는데, 사진은 JSON이 아닙니다. 이 챕터에서는 사진을 **글자로 바꿔서** JSON 안에 태워 보내는 가장 단순한 방법을 손으로 만들어 보고, 그 단순함이 주는 편안함과 동시에 왜 이 방식이 10MB 앞에서 무너지는지 직접 눈으로 확인합니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

챕터 4의 실습 레포는 하나입니다. Spring Boot 한 덩어리에 H2 인메모리 DB가 같이 달려 있습니다.

레포	용도	주소
spring-base64	Spring Boot + H2 + 로컬 `uploads` 폴더	`github.com/metacoding-11-spring-reference/spring-base64`

터미널에서 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/spring-base64.git
cd spring-base64
```

파일 구조는 이렇습니다.

spring-base64 디렉토리

```
spring-base64/
├── build.gradle
├── src/main/
│   ├── java/com/metacoding/spring_base64/
│   │   ├── _core/config/
│   │   │   ├── WebConfig.java              # [실습] /uploads/** 정적 리소스 매핑
│   │   │   └── CorsConfig.java             # [참고] 브라우저 호출용 CORS
│   │   └── image/
│   │       ├── ImageController.java        # [실습] /upload · /list · /{id}
│   │       ├── ImageService.java           # [실습] Base64 디코드 → 저장 → DB
│   │       ├── ImageEntity.java            # [실습] image_tb 엔티티
│   │       ├── ImageRequest.java           # [실습] UploadDTO (fileName·fileDa
ta)
│   │       ├── ImageResponse.java          # [실습] DTO (id·uuid·fileName·url·
createdAt)
│   │       └── ImageRepository.java        # [참고] JpaRepository
│   └── resources/
│       └── application.properties          # [실습] static-locations=file:uplo
ads/
└── uploads/                                # [참고] 업로드된 파일이 떨어지는 로
컬 폴더
```

> **이 레포는 챕터 3의 레포와 별개입니다.** 두 Spring 서버 + Nginx + Redis 네 컨테이너는 이미 `docker compose down`으로 정리하고 내려 두셨다고 가정합니다. 이번 챕터는 **단일 Spring Boot**에만 집중합니다. 인프라를 다시 얹는 일은 챕터 5부터 이어집니다.

2. 실습 환경 구축

이번 챕터에 새로 깔 것은 없습니다. 챕터 1에서 설치해 둔 JDK 21과 Gradle이면 충분합니다. Postman은 아직 설치하지 않았다면 postman.com에서 받아 둡니다. 호출 결과 JSON을 예쁘게 보여 주는 도구가 하나쯤은 필요합니다.

터미널 실습 환경 확인

```
java --version
./gradlew --version
```

3. 사용할 구성 요소

챕터 4에 새로 얹히는 재료는 네 가지입니다.

재료	역할
`spring-boot-starter-web`	REST API 엔드포인트와 정적 리소스 서빙
`spring-boot-starter-data-jpa`	엔티티·리포지토리로 DB에 파일 메타를 기록
`com.h2database:h2`	인메모리 DB. 콘솔(`/h2-console`)까지 기본 제공
`java.util.Base64`	JDK 표준 Base64 인코더·디코더 (별도 라이브러리 없음)

왜 Base64부터 손으로 해 보는가

원본 바이너리를 `multipart/form-data`로 받는 방법이 더 효율적이라는 걸 이 챕터 끝에서 숫자로 확인하게 됩니다. 그런데도 Base64를 먼저 손으로 다루는 이유는 두 가지입니다. 첫째, JSON API만 쓰는 환경(모바일 SDK·서드파티 훅·프록시로 막힌 사내망 등)에서는 **업로드도 JSON**

한 줄로 끝내고 싶은 요구가 실제로 자주 나옵니다. 둘째, Base64라는 방식의 동작과 한계를 숫자로 체감해야 챕터 5의 **Presigned URL + S3** 가 왜 필요한 기술인지가 선명해집니다. 방법을 알고 버리는 것과, 모르고 다른 걸 배우는 것은 다릅니다.

4. 실습 순서

이번 챕터의 절은 이 순서로 진행합니다.

1. `4.1` . 사진을 JSON에 담으려던 순간의 벽
2. `4.2` . 엽서로 사진을 적어 보낸다는 생각
3. `4.3` . 저장 경로와 엔티티 설계
4. `4.4` . 요청·응답 DTO 설계
5. `4.5` . 업로드 API 구현 (Base64 → 파일 → DB)
6. `4.6` . 정적 리소스 매핑과 조회 API
7. `4.7` . Postman으로 전체 흐름 확인
8. `4.8` . 10MB 사진으로 한계를 부딪쳐 보기

1에서 문제를 만난 뒤 2에서 비유로 길을 잡습니다. 3~6에서 코드로 내려놓고, 7에서 Postman으로 검증한 뒤, 8에서 이 방식의 바닥을 직접 두드려 봅니다.

4.1 사진을 JSON에 담으려던 순간

챕터 3의 마지막, 네 컨테이너를 전부 내리고 난 오후였습니다.

모니터 한쪽에는 아직 `docker compose down` 의 마지막 로그가 희미하게 떠 있었습니다. 오픈이는 노트북 받침대를 한 단 내리고, 새 프로젝트를 열었습니다.

옆자리 동료가 마우스 휠을 딱딱 굴리다가 의자를 반 바퀴 돌렸습니다.

동료 : "프로필 사진 말이에요. 지금 우리 화면에 기본 아바타만 뜨잖아요. 사진 바꾸는 기능 언제 해요?"

오픈이 : "아, 그거요. 오늘 해 볼게요."

간단하겠지.

오픈이는 Postman을 열었습니다. 평소처럼 `POST /upload`에 JSON을 실어 보내면 끝일 것 같았습니다. Body 탭에서 raw·JSON을 고르고, 텍스트 영역에 손으로 타이핑했습니다.

```
{
  "name": "profile",
  "image": ???
}
```

???

여기서 손이 멈췄습니다. `name`은 문자열이니까 괄호 안에 적어 넣으면 끝이었는데, `image`는 어디서 적어야 할지를 몰랐습니다. 사진은 제 책상에 PNG 파일로 들어 있었습니다. 파일 경로를 적어야 하나 싶어 `"C:/Users/.../profile.png"`라고 써 봤지만, 그 문자열은 **내 컴퓨터의 경로**였지 사진 자체가 아니었습니다.

JSON은 글자밖에 못 담는다.

JSON의 값은 문자열·숫자·불·객체·배열뿐이었습니다. 그 어디에도 "바이너리 파일"이라는 타입은 없었습니다. 사진은 0101로 가득한 바이트 덩어리였고, JSON은 글자로 된 그릇이었습니다.

Postman의 Body 탭을 다시 들여다봤습니다. `form-data`라는 탭이 따로 있었습니다. 거기서 Key를 `file`로 두고, Type을 `File`로 바꾸면 실제 파일을 고를 수 있었습니다. Content-Type이 `multipart/form-data`로 바뀌었습니다.

이걸로 하면 되겠네.

Spring 쪽에서 `@RequestParam MultipartFile file`로 받으면 된다는 건 어렴풋이 알고 있었습니다. 컨트롤러에 코드를 한 줄 쓰려다가, 손이 또 멈췄습니다.

근데 왜 지금까지 우리는 다 JSON으로만 주고받았지.

팀장이 몇 번이나 말했던 규칙이 있었습니다. **프런트엔드·모바일·외부 연동까지 한 번에 지원하려면 API는 전부 JSON 하나로 통일한다.** 로그인도 JSON, 사용자 정보 수정도 JSON, 게시글도 JSON이었습니다. 그런데 사진 업로드만 갑자기 `multipart/form-data`로 빠지면, 프런트 쪽에서는 이 요청 하나만 따로 처리해야 했습니다. 모바일 SDK 중에는 multipart를 덜 친절하게 지원하는 것들도 있었습니다.

하나만 다른 건 싫은데.

그때 팀장이 지나가다 멈췄습니다.

팀장 : "사진 올리는 거 하네요? 예전에 해 봤던 방식 하나 있어요. 폰으로 사진 찍어서 카톡으로 보낼 때 말이에요. 그거 뒤에서 어떻게 갈 것 같아요?"

오픈이 : "음.... 파일로 가지 않나요?"

팀장 : "클라이언트 쪽에서는 파일처럼 보이죠. 그런데 네트워크 타면 결국 글자예요. 사진을 글자로 치환해서 보내는 방식이 예전부터 있었어요."

오픈이 : "글자로요?"

팀장 : "엽서에 사진을 통째로 붙일 수는 없잖아요. 근데 사진을 엄청 길게 받아써서 엽서에 적으면, 받은 쪽에서 그걸 다시 사진으로 복원할 수는 있죠."

팀장은 그 말만 남기고 회의실로 들어갔습니다.

사진을 글자로 받아쓴다.

오픈이는 수첩을 폈습니다. 아까 네 번째 시도에서 `form-data`로 넘어갔던 걸 지우고, 원래 자리인 JSON으로 돌아왔습니다. JSON이 담을 수 있는 건 글자뿐이니, **사진을 글자로 만들기만 하면** 여기에 태울 수 있다는 이야기였습니다.

문제는 방법이었습니다. 바이트는 0부터 255까지의 값을 가지는데, JSON 문자열에 그대로 집어넣으면 제어 문자·따옴표·백슬래시가 섞여 들어가서 파싱이 깨집니다. 그러니까 **JSON이 안전하게 담을 수 있는 글자**로만 바이트를 옮겨 적어야 했습니다.

제한된 글자 64개 같은 걸로 다시 쓰는 규약이 있을 거 같은데.

검색창에 "binary to text encoding"이라고 쳤습니다. 첫 번째 결과가 **Base64** 였습니다.

4.2 엽서로 사진을 적어 보낸다

Base64는 64개의 글자(A-Z, a-z, 0-9, `+`, `/`) 만 써서 모든 바이트를 다시 쓰는 방식이었습니다. 규칙은 단순했습니다. 8비트짜리 바이트 세 개(총 24비트)를 가져와서, 6비트씩 네 조각으로 잘라 64개 글자 중 하나로 바꿉니다. 사진의 모든 바이트가 이 방식으로 글자들로 쭉 늘어섭니다.

그럼 JSON에 그대로 태울 수 있겠구나.

[GEMINI PROMPT: 왼쪽에 고양이 사진 한 장(PNG)이 놓여 있고, 그 옆에 큰 화살표가 엽서 모양 상자로 이어진다. 엽서 상자 안에는 "iVBORw0KGgoAAAANSUhEUgAA..."처럼 Base64 문자열이 여러 줄에 걸쳐 빽빽이 적혀 있다. 엽서 아래에는 작은 손글씨로 "사진을 글자로 받아썼습니다"가 적혀 있다. 깔끔한 일러스트, 흰 배경, 인디고·오렌지 악센트.]

그림 4-1. 사진을 엽서 한 장에 글자로 받아쓰는 방식이 Base64입니다

이 방식의 약점은 따로 있었습니다. 3바이트가 4글자로 늘어나니까, **크기가 약 33퍼센트 불어난다**는 점이었습니다. 1MB짜리 사진은 Base64로 바꾸는 순간 약 1.33MB가 되었습니다. 엽서로 사진을 받아쓰면 사진 원본보다 엽서 더미가 더 두꺼워진다는 얘기였습니다. 오픈이는 그 비용이 얼마나 커질지 아직 감이 없었지만, 일단은 JSON 하나로 사진을 보낼 수 있다는 것이 더 크게 보였습니다.

그림 4-2. 사진 한 장이 서버에 닿아 다시 화면에 뜨기까지



요청은 글자로 오고, 응답은 URL로 나갑니다. 사진은 서버의 `uploads` 에만 한 번 존재합니다

오픈이는 수첩에 결정할 것들을 정리했습니다.

— 설계 정리 —

1. 요청
   - JSON 한 덩어리. 필드는 `fileName` · `fileData` 둘
   - `fileData`는 Base64 문자열
2. 서버
   - 문자열을 디코드해 바이트로 만든다
   - UUID를 앞에 붙여 새 파일명으로 `uploads/`에 저장
   - 원본 이름·UUID·공개 URL·생성 시각을 DB에 기록
3. 응답
   - 저장된 URL을 포함한 JSON
   - 그 URL을 브라우저에 붙이면 그대로 사진이 떠야 한다
4. 확인할 것
   - Postman 흐름이 끝에서 끝까지 돌아가는가
   - 사진이 커질수록 요청이 얼마나 무거워지는가

이제 이 설계를 코드로 옮기겠습니다.

4.3 저장 경로와 엔티티 설계

4.3.1 업로드 저장 경로

실제 사진 파일은 DB가 아니라 **로컬** `uploads` **폴더**에 떨어집니다. DB에는 그 파일의 경로와 메타데이터만 들어갑니다. 이 분리가 이번 챕터의 핵심 설계입니다.

레포 루트에 `uploads/` 폴더가 이미 있는지 확인합니다. 없으면 빈 폴더를 만들어 둡니다.

터미널 uploads 폴더 확인

```
ls -d uploads 2>/dev/null || mkdir uploads
```

`src/main/resources/application.properties`를 열고 **정적 리소스 경로**가 `uploads/`를 가리키는지 확인합니다.

실습 1 src/main/resources/application.properties. 정적 리소스 경로

```
spring.web.resources.static-locations=file:uploads/
```

이 한 줄이 "Spring이 제공하는 정적 파일의 실제 소스는 프로젝트 폴더 안의 `uploads/`"라는 선언입니다. 이 값은 4.6절의 `WebConfig`와 짝을 이룹니다. 여기서는 어디에서 읽을지만, 거기에서는 어떤 URL을 그 폴더로 매핑할지를 정합니다.

4.3.2 엔티티 설계

DB에 찍히는 한 행이 어떻게 생겼는지부터 잡습니다. 엔티티는 DB 테이블과 1:1로 짝지어지는 자바 클래스입니다. 우리가 저장할 정보는 네 가지 + ID였습니다.

`src/main/java/com/metacoding/spring_base64/image/ImageEntity.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 2 src/main/java/com/metacoding/spring_base64/image/ImageEntity.java. image_tb 매핑

```
@Entity
@Table(name = "image_tb")
public class ImageEntity {

    // TODO: DB 한 행이 가질 다섯 필드를 채웁니다
    // 1. id — PK. DB가 AUTO_INCREMENT로 채워 준다
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // 2. uuid — 파일명 충돌을 막는 고유 값
    private String uuid;

    // 3. fileName — 실제로 디스크에 저장된 이름 (UUID + 확장자)
    private String fileName;

    // 4. url — 공개 URL. 응답에 그대로 실어 보낸다
    private String url;

    // 5. createdAt — 업로드 시각
    private LocalDateTime createdAt;
}
```

한 줄씩 풀어 보겠습니다. `id`는 DB가 알아서 1, 2, 3으로 올려 줍니다. `uuid`는 같은 이름의 사진이 여러 번 올라와도 덮어쓰지 않기 위해 필요합니다. 사람이 `cat.png`라는 이름으로 두 번 올려도 실제 파일은 서로 다른 UUID로 저장됩니다. `fileName`은 디스크에 남은 이름(`{uuid}.png`)이고, `url`은 `/uploads/{fileName}` 형태로 외부에 노출될 경로입니다. URL을 미리 만들어 DB에 저장해 두면 응답에서 한 번 더 가공할 필요가 없습니다.

필드	값의 형태	저장 이유
`id`	`1`, `2`, `3`...	상세·수정·삭제 API의 기준
`uuid`	`c5b8f37c-e767-46b1-97fd-e2d67bd79dff`	파일명 충돌 방지 + 외부에 ID를 감추고 싶을 때의 대안 키
`fileName`	`{uuid}.png`	디스크에 실제로 저장된 이름
`url`	`/uploads/{uuid}.png`	응답에 그대로 찍히는 공개 경로
`createdAt`	`2025-12-25T20:57:32.436972`	목록 정렬·감사 로그

4.4 요청·응답 DTO 설계

엔티티는 DB의 한 행을 표현하는 반면, DTO는 API의 한 요청·한 응답을 표현합니다. 둘을 섞으면 DB 스키마가 바뀔 때마다 API 응답이 같이 바뀌어 버립니다. 그래서 **요청 DTO와 응답 DTO를 따로** 두고, 서비스 안에서 엔티티 ↔ DTO 변환을 합니다.

4.4.1 요청 DTO

`src/main/java/com/metacoding/spring_base64/image/ImageRequest.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 3 src/main/java/com/metacoding/spring_base64/image/ImageRequest.java. 업로드 요청

```
public class ImageRequest {

    // TODO: 업로드 요청 한 건을 담을 record를 만듭니다
    // 1. fileName — 원본 파일명 ("cat.png"). 확장자를 뽑는 근거
```

```
    // 2. fileData — Base64로 인코딩된 사진의 본체
    public record UploadDTO(
            String fileName,
            String fileData
    ) {}
}
```

이 구조가 Postman에서 보내는 JSON과 정확히 짝지어집니다.

```
{
  "fileName": "cat.png",
  "fileData": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

`fileData`는 그냥 긴 문자열입니다. 끝에 `=` 패딩이 한두 개 붙는 경우도 있지만, Spring은 그걸 그대로 문자열로 받아 내려 줍니다.

4.4.2 응답 DTO

`src/main/java/com/metacoding/spring_base64/image/ImageResponse.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 4 src/main/java/com/metacoding/spring_base64/image/ImageResponse.java. 업로드·조회 응답

```
public class ImageResponse {

    // TODO: 업로드·조회 공용 응답 record를 만듭니다
    public record DTO(
            Long id,
            String uuid,
            String fileName,
            String url,
            LocalDateTime createdAt
    ) {
        // 1. 엔티티를 응답 DTO로 변환하는 정적 팩토리
        public static DTO fromEntity(ImageEntity e) {
            return new DTO(
                    e.getId(),
                    e.getUuid(),
                    e.getFileName(),
                    e.getUrl(),
                    e.getCreatedAt()
```

```
            );
        }
    }
}
```

`fromEntity` 하나가 있으면 서비스 쪽 코드가 확 가벼워집니다. `return ImageResponse.DTO.fromEntity(saved);` 한 줄로 "DB에 저장한 엔티티를 응답 모양으로 바꿔서 돌려준다"가 끝납니다. 업로드·단건 조회·목록 조회가 모두 같은 DTO 하나로 응답합니다.

4.5 업로드 API 구현

이 절은 네 조각으로 나누어 씁니다. Base64 문자열을 바이트로 되돌리고 → UUID로 새 파일명을 만들고 → 디스크에 쓴 뒤 → DB에 한 행을 심는 순서입니다. 네 조각이 모두 `ImageService.java` 한 파일 안에서 일어납니다.

4.5.1 Base64 디코딩

사진 원본은 글자로 변환돼서 들어왔습니다. 이 글자를 다시 바이트 배열로 되돌리는 것이 첫 단계입니다. 이 자리가 4.2절에서 엽서에 적힌 글자를 다시 사진으로 복원하는 지점입니다.

`src/main/java/com/metacoding/spring_base64/image/ImageService.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 5 src/main/java/com/metacoding/spring_base64/image/ImageService.java. Base64 디코딩

```
@Transactional
public ImageResponse.DTO upload(ImageRequest.UploadDTO uploadDTO) {
    // TODO: 1. Base64 문자열을 원본 바이트로 복원한다
    byte[] fileBytes = Base64.getDecoder().decode(uploadDTO.fileData());

    // 2. ... (다음 절에서 파일명 만들기)
    // 3. ... (다음 절에서 디스크 저장)
    // 4. ... (다음 절에서 DB 기록)
}
```

`Base64`는 `java.util.Base64`입니다. 별도 라이브러리가 필요하지 않습니다. `getDecoder().decode(...)` 한 줄이 긴 문자열을 원본 바이트로 되돌립니다. 이 `fileBytes`가 곧 디스크에 떨어질 실제 사진입니다.

Data URI 스키마를 섞어 보내도 될까

프런트엔드에서 `FileReader.readAsDataURL()`로 읽으면 `...` 같은 접두어가 붙은 문자열이 나옵니다. 여기서 `` 부분은 **Base64 본체가 아니라 MIME 헤더**입니다. Spring 쪽은 순수 Base64 본체만 기대하기 때문에, 프런트에서 콤마(`,`) 뒤의 본체만 잘라 보내거나, 서버에서 `split(",")` 후 뒤쪽만 디코딩하도록 방어 코드를 두는 방식이 있습니다. 이 챕터의 예제는 Postman으로 본체만 붙여 넣는 것을 전제로 합니다.

4.5.2 파일명과 확장자 만들기

사람이 올린 원본 이름(`cat.png`)을 그대로 디스크에 쓰면 같은 이름의 파일이 올라왔을 때 덮어써집니다. UUID를 새 파일명으로 쓰고, 확장자만 원본에서 뽑아 붙입니다.

실습 6 src/main/java/com/metacoding/spring_base64/image/ImageService.java. 파일명 생성

```
// 2. 충돌 없는 새 파일명을 만든다
String uuid = UUID.randomUUID().toString();

// 2-1. 원본 이름에서 마지막 점 뒤를 확장자로 뽑는다 ("cat.png" → "png")
int dotIndex = uploadDTO.fileName().lastIndexOf('.');
String fileExtension = uploadDTO.fileName()
        .substring(dotIndex + 1)
        .trim()
        .toLowerCase();

// 2-2. 저장될 실제 파일명 ("{uuid}.png")
String savedFileName = uuid + "." + fileExtension;
```

`UUID.randomUUID()`는 `c5b8f37c-e767-46b1-97fd-e2d67bd79dff` 같은 36자리 문자열을 매번 새로 만듭니다. `lastIndexOf('.')`로 마지막 점 위치를 잡는 이유는 `my.cat.2024.png`처럼 점이 여러 개 있는 파일명도 맨 뒤의 확장자만 가져오기 위함입니다.

4.5.3 로컬 디스크 저장

이제 디코딩된 바이트 배열을 `uploads/` 폴더에 파일로 씁니다.

실습 7 src/main/java/com/metacoding/spring_base64/image/ImageService.java. 디스크 저장

```
// 3. 로컬 uploads 폴더에 바이트를 파일로 쓴다
Path uploadDir = Paths.get("uploads");
Path filePath = uploadDir.resolve(savedFileName);
Files.write(filePath, fileBytes);
```

`Paths.get("uploads")`는 프로젝트 **실행 디렉토리 기준의 상대경로**입니다. IntelliJ에서 실행하면 보통 프로젝트 루트가 작업 디렉토리가 됩니다. 실무에서는 OS 경로 구분자 문제(`/` vs `\`)와 실행 위치 차이 때문에 **절대경로 + 프로퍼티로 관리**로 바꾸지만, 이 챕터는 한 화면에서 읽을 수 있는 단순함을 택했습니다. `Files.write`는 파일이 없으면 만들고, 있으면 덮어씁니다. UUID로 파일명을 유일하게 했기 때문에 덮어쓰일 걱정은 없습니다.

4.5.4 DB 기록과 응답

파일을 썼으니 이제 그 사실을 DB에 기록합니다. 공개 URL도 이 자리에서 만들어 엔티티에 박아 둡니다.

실습 8 src/main/java/com/metacoding/spring_base64/image/ImageService.java. DB 기록과 응답

```
    // 4. URL을 만들고 엔티티에 담아 DB에 한 행을 심는다
    String publicUrl = "/uploads/" + savedFileName;

    ImageEntity entity = ImageEntity.builder()
            .uuid(uuid)
            .fileName(savedFileName)
            .url(publicUrl)
            .createdAt(LocalDateTime.now())
            .build();

    ImageEntity saved = imageRepository.save(entity);

    // 5. 엔티티를 응답 DTO로 변환해 돌려준다
    return ImageResponse.DTO.fromEntity(saved);
}
```

`publicUrl`은 4.3.1의 `spring.web.resources.static-locations=file:uploads/`와 4.6절에서 쓸 `WebConfig`의 `/uploads/**` 매핑에 의존합니다. 이 세 조각이 한 세트로 맞물려서, 응답의 `url` 한 줄을 브라우저 주소창에 그대로 붙이면 사진이 뜹니다.

`ImageService`는 `@Service`가 달려 있고, 생성자 주입으로 `ImageRepository`를 받습니다.
`ImageRepository`는 `JpaRepository<ImageEntity, Long>`을 상속한 인터페이스입니다.
`save`, `findById`, `findAll`이 기본으로 들어 있어 이번 챕터에서는 이 정도만 씁니다.

호출	겉으로 하는 일	속으로 일어나는 일
`Base64.getDecoder().decode(fileData)`	문자열을 바이트로	4글자씩 읽어 3바이트로 환원
`Files.write(filePath, fileBytes)`	디스크에 파일로 쓰기	NIO가 OS의 파일 핸들을 열어 바이트를 기록
`imageRepository.save(entity)`	DB에 한 행 저장	JPA가 `INSERT INTO image_tb ...` 쿼리를 발행

4.6 정적 리소스 매핑과 조회 API

4.6.1 /uploads/** 경로 매핑

4.3.1에서 `file:uploads/`를 한 번 지정했지만, `WebConfig`에서 한 번 더 선언해서 "`/uploads/**`로 들어오는 요청은 이 폴더에서 찾아 응답하라"를 분명하게 써 둡니다.

`src/main/java/com/metacoding/spring_base64/_core/config/WebConfig.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 9 src/main/java/com/metacoding/spring_base64/_core/config/WebConfig.java. 정적 리소스 매핑

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        // TODO: /uploads/** 요청을 로컬 uploads 폴더로 연결한다
        registry.addResourceHandler("/uploads/**")
                .addResourceLocations("file:uploads/");
    }
}
```

`addResourceHandler("/uploads/**")`는 "이 URL 패턴이 오면"이고, `addResourceLocations("file:uploads/")`는 "이 실제 폴더에서 찾아라"입니다. `file:` 접두어는 **디스크의 경로**라는 의미입니다. `classpath:` 접두어를 쓰면 JAR 안의 정적 리소스를 뒤지지만, 업로드된 파일은 빌드 시점에 JAR 안에 있지 않으므로 `file:`이 맞습니다.

4.6.2 컨트롤러

업로드·단건 조회·목록 조회 세 엔드포인트를 하나의 컨트롤러에 둡니다.

`src/main/java/com/metacoding/spring_base64/image/ImageController.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 10 src/main/java/com/metacoding/spring_base64/image/ImageController.java. 세 엔드포인트

```
@RestController
@RequiredArgsConstructor
public class ImageController {

    private final ImageService imageService;

    // TODO: 업로드·단건·목록 세 엔드포인트를 매핑합니다
    // 1. POST /upload — Base64 JSON을 받아 파일로 저장
    @PostMapping("/upload")
    public ImageResponse.DTO upload(@RequestBody ImageRequest.UploadDTO dto) {
        return imageService.upload(dto);
    }

    // 2. GET /{id} — DB에 심은 한 행을 id로 조회
    @GetMapping("/{id}")
    public ImageResponse.DTO getImageDetail(@PathVariable Long id) {
        return imageService.findById(id);
    }

    // 3. GET /list — 전체 목록
    @GetMapping("/list")
    public List<ImageResponse.DTO> getAllImages() {
        return imageService.listAll();
    }
}
```

`findById`와 `listAll`은 `ImageService` 안에서 `imageRepository.findById(id)`와 `imageRepository.findAll()`을 감싸서 `ImageResponse.DTO.fromEntity(...)` 변환만 추가한 단순 메서드입니다. 리포지토리를 컨트롤러에서 직접 건드리지 않고 서비스 한 겹을 거치는 것이 이

레포의 규칙입니다.

이제 세 조각(저장 경로·매핑·컨트롤러)이 맞물렸으니, Postman으로 실제로 돌려 보겠습니다.

4.7 Postman으로 전체 흐름 확인

서버를 띄웁니다.

터미널 실험 4-1 실행. Spring 서버 기동

```
./gradlew bootRun
```

`Started SpringBase64Application in 3.xxx seconds` 로그가 찍히면 준비 완료입니다.

[CAPTURE NEEDED: ./gradlew bootRun 실행 후 "Started SpringBase64Application" 메시지와 "Tomcat started on port 8080"이 찍힌 터미널 화면]

4.7.1 사진을 Base64로 바꾸기

프로필용으로 쓸 작은 PNG 한 장(100KB 이하의 고양이 사진이면 충분합니다)을 준비합니다. 사진을 Base64로 바꿔 주는 도구는 여러 가지인데, 이 챕터는 브라우저에서 바로 쓸 수 있는 웹 도구를 씁니다.

1. https://www.base64decode.org/에 접속합니다
2. 파일 드래그 영역에 준비한 PNG를 놓고 `ENCODE` 버튼을 누릅니다
3. 아래쪽 텍스트 영역에 긴 문자열이 찍힙니다. 이 전체를 복사합니다

[CAPTURE NEEDED: base64decode.org에서 PNG 업로드 후 ENCODE 버튼을 눌러, 하단 텍스트 영역에 "iVBORw0KGgoAAAANSUhEUgAA..."로 시작하는 Base64 문자열이 한 가득 찍힌 화면]

명령줄로도 같은 걸 할 수 있습니다

GUI 도구 없이 터미널에서만 돌리고 싶다면 macOS·Linux에서는 `base64 -i cat.png > cat.b64`, Windows PowerShell에서는 `[Convert]::ToBase64String([IO.File]::ReadAllBytes("cat.png"))`로 같은 문자열을 얻을 수 있습니다. 결과는 동일합니다. 이 챕터는 복사·붙여넣기의 편의를 위해 웹 도구를 쓸 뿐입니다.

4.7.2 업로드

Postman을 열고 새 요청을 만듭니다.

- 메서드: `POST`
- URL: `http://localhost:8080/upload`
- Body 탭 → raw → JSON

Body에 이 JSON을 붙입니다. `fileData` 값에는 방금 복사한 긴 Base64 문자열을 통째로 넣습니다.

```
{
  "fileName": "cat.png",
  "fileData": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

[CAPTURE NEEDED: Postman에서 POST http://localhost:8080/upload에 Body(raw, JSON)로 fileName과 긴 fileData가 들어간 JSON을 붙인 화면]

`Send`를 누릅니다. 200 OK와 함께 응답이 돌아옵니다.

[CAPTURE NEEDED: Postman 응답 창. 200 OK 상태에 id, uuid, fileName(c5b8f37c-...png), url(/uploads/c5b8f37c-...png), createdAt이 찍힌 JSON]

응답의 `url` 값이 핵심입니다. 이 문자열 앞에 `http://localhost:8080`만 붙이면 브라우저에 바로 붙일 수 있는 주소가 됩니다.

4.7.3 브라우저로 확인

`http://localhost:8080/uploads/{응답의 fileName}`을 주소창에 붙입니다. 방금 올린 사진이 그대로 뜹니다.

[CAPTURE NEEDED: 브라우저 주소창에 http://localhost:8080/uploads/c5b8f37c-...png를 입력해 고양이 사진이 그대로 렌더링된 화면]

이 순간이 4.2절 다이어그램의 마지막 단계(Browser가 URL로 사진을 그대로 받는 장면)가 그대로 실현되는 지점입니다. 요청은 글자로 갔지만, 응답은 **URL 한 줄**로 왔고, 사진은 서버의 `uploads` 폴더 한 곳에만 남았습니다.

4.7.4 DB 확인과 조회 API

H2 콘솔을 열어 실제로 DB에 한 행이 심어졌는지 확인합니다.

`http://localhost:8080/h2-console`
로그인 화면의 JDBC URL은 `application.properties`에 설정된 값(기본 `jdbc:h2:mem:testdb`)을 그대로 씁니다.

[CAPTURE NEEDED: http://localhost:8080/h2-console에서 `SELECT * FROM image_tb` 쿼리를 실행한 결과. 한 행에 id=1, uuid, file_name, url, created_at이 찍힌 화면]

Postman으로 돌아와 목록과 단건을 확인합니다.

- `GET http://localhost:8080/list`
- `GET http://localhost:8080/1`

[CAPTURE NEEDED: Postman에서 GET /list 응답. 배열 안에 방금 올린 한 건이 들어간 JSON]

[CAPTURE NEEDED: Postman에서 GET /1 응답. id=1 한 건이 단일 객체로 내려오는 JSON]

세 엔드포인트가 모두 같은 `ImageResponse.DTO` 모양을 응답합니다. 프런트엔드는 업로드·목록·상세를 같은 파싱 함수로 처리하면 됩니다.

4.8 10MB 사진으로 한계를 부딪쳐 본다

동작 확인을 끝냈으니, 이제 이 방식의 **바닥**을 두드릴 차례였습니다. 프로필 사진은 보통 1MB 이하지만, 사용자가 최신 스마트폰으로 찍은 원본을 그대로 올리는 경우가 흔했습니다. 요즘 폰 카메라는 한 장에 8~12MB 정도가 나왔습니다. 그걸 그대로 올려 보면 어떻게 되는지 확인해 보기로 했습니다.

동료가 마침 그 쪽으로 말을 걸었습니다.

동료 : "저 방금 진짜로 제 셀카 올려봤어요. 10MB쯤 되는 거요. 근데 Postman에서 Send 누르니까 몇 초 멍하니 있다가 답 오네요."

오픈이 : "얼마나요?"

동료 : "4초, 5초 정도요. 그리고 서버 쪽 콘솔도 뭔가 낑낑대는 느낌이에요."

오픈이도 똑같이 해 봤습니다. 10MB 원본 사진을 Base64로 바꾸니 텍스트 파일 크기가 **약 13.3MB**로 불어 있었습니다. 33퍼센트 증가라는 말이 그제야 숫자로 잡혔습니다.

그림 4-3. 원본 vs Base64 크기 비교

프로필(100KB)	Base64 ≈ 133KB · 요청 크기 무시할 만함 · 응답 거의 즉시
고화질(1MB)	Base64 ≈ 1.33MB · 요청이 JSON이라 파싱 비용 발생 · 체감 가능한 지연
폰 원본(10MB)	Base64 ≈ 13.3MB · 요청 바디 비대 · 힙에 큰 String 상주 · 3~5초 지연

동일 사진을 올리는 것만으로 JSON 요청은 33퍼센트 더 무거워집니다

한계가 한두 가지가 아니었습니다.

첫째, **요청 바디 자체가 무거워집니다.** JSON으로 감싼 Base64는 원본 바이너리보다 크고, Spring의 Jackson이 이 긴 문자열 필드를 JSON 파싱해 String 객체 하나로 만들어야 합니다. 10MB짜리 원본이면 JSON 파싱 단계에서 13MB짜리 String이 힙에 한 번 올라갑니다. 동시 업로드가 몇 건만 몰려도 힙이 금방 뜨거워집니다.

둘째, **디코딩 후에도 원본 바이트 배열이 또 힙에 올라갑니다.** Base64 문자열(13MB 클래스 `char[]`·`byte[]`) + 디코딩 결과(10MB `byte[]`)가 한 요청에 동시에 잡혀 있습니다. 서버 입장에서는 요청 하나당 25MB 안팎의 메모리를 요구받는 셈입니다.

셋째, **Spring Boot의 기본 `max-http-form-post-size`나 `max-request-size`에 걸립니다.** 기본값은 보통 1MB~10MB 수준이라, 13MB 요청은 한도 설정을 올리지 않으면 그대로 튕겨 나옵니다. 한도를 키우는 건 임시 처방이고, 그 한도가 곧 **서비스 전체가 감당해야 할 업로드 상한**이 됩니다.

넷째, **DB까지 바이너리를 태울 생각을 하면 악몽이 됩니다.** 이 챕터는 이미지를 파일시스템에 넣고 경로만 DB에 기록하는 구조를 택했습니다. 만약 대신 **DB의 TEXT/BLOB 컬럼**에 Base64 문자열이나 바이너리를 통째로 박아 넣는 설계를 택한다면, 테이블 하나가 수 기가로 금세 불어나고 풀 스캔 한 번에 DB가 비명을 지릅니다. `LIKE` 검색·인덱스는 사실상 무의미해집니다.

다섯째, **네트워크 비용이 그대로 늘어납니다.** 업로드뿐 아니라, 이미지를 Base64로 응답에 다시 실어 내려주는 순간 같은 33퍼센트가 다시 한 번 붙습니다. 트래픽이 원본 대비 1.33배 × 2회(업·다운) 수준으로 늘어날 수 있습니다.

이 방식으로 10만 명의 프로필 사진을 받을 수 있을까.

오픈이는 커피를 한 모금 마시고 수첩에 정리했습니다.

```
— 한계 정리 —

1. 요청이 무거워진다
   · 원본 대비 약 33퍼센트 증가
   · JSON 파싱 단계에서 큰 String이 힙에 상주

2. 서버가 바빠진다
   · 디코딩 결과 byte[]가 또 힙에 올라간다
   · 동시 업로드가 몰리면 OOM 위험이 현실이 됨

3. DB가 위험하다
   · 파일 자체를 DB에 박는 설계는 빠르게 무너진다
   · 경로만 DB에 두는 이번 설계도 트래픽 급증에 대응이 어렵다

4. 업로드 한도가 곧 서비스 한계
   · max-request-size를 키울수록 공격 벡터도 넓어진다
   · 이 구조 위에서 10MB, 50MB, 100MB는 현실적이지 않다
```

동료가 다시 의자를 돌렸습니다.

동료 : "저희 서비스에서 진짜 큰 이미지도 받나요? 동영상 프로필 같은 거 나중에 붙으면요?"

오픈이 : "그때는 이 방식으로 못 받아요."

동료 : "그럼요?"

팀장 : "업로드를 서버가 직접 받는 방식부터 바꿔야 할 거예요. 사진을 꼭 내 서버를 거쳐서 받을 필요는 없거든요. 사용자가 직접 저장소에 올리게 하고, 서버는 주소만 내주는 방식."

오픈이 : "저장소에 직접이요?"

팀장 : "S3 같은 클라우드 저장소에요. 서버는 안 거쳐요. '이 주소로 이 시간 안에 올려' 하고 권한 찍힌 링크만 내주는 거예요."

주소만 내준다.

오픈이의 머릿속에 금방 그림이 그려지지 않았습니다. 내 서버가 사진을 받지 않는다면, 사진은 어디에 떨어지고, 그 떨어진 사진은 어떻게 썸네일·리사이즈를 거치며, 서버는 언제 그 사실을 알게 될까요. 파이프라인이 한 덩어리에서 여러 덩어리로 쪼개지는 순간이 다가오고 있었습니다.

주소만 내주는 업로드. Lambda로 변환. 서버는 완료 알림.

다음 챕터의 문이 열리고 있었습니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
사진을 글자로 받아쓰기	Base64 인코딩	바이너리 데이터를 64개의 ASCII 글자(A-Z, a-z, 0-9, +, /)만으로 표현하는 인코딩 방식. RFC 4648
엽서 위 긴 글자를 다시 사진으로	Base64 디코딩	Base64 문자열을 원본 바이트 배열로 되돌리는 과정. JDK는 `java.util.Base64.getDecoder()` 가 표준 제공
33퍼센트 불어남	인코딩 오버헤드	3바이트 → 4글자로 변환되어 데이터 크기가 약 4/3로 증가하는 특성. 패딩(`=`) 포함
사진 앞의 프로토콜 머리말	Data URI 스키마	`data:[<mediatype>][;base64],<data>` 형식. 브라우저가 문자열에서 곧바로 리소스를 렌더링하도록 하는 규격 (RFC 2397)
폼으로 사진 올리기	multipart/form-data	바이너리 파일과 필드를 한 요청에 담기 위한 HTTP 요청 본문 형식. 업로드 전용 표준
서버가 가진 전용 글자 그릇	엔티티	JPA에서 DB 테이블 한 행에 매핑되는 자바 클래스. `@Entity` 로 선언
요청·응답 전용 포장지	DTO (Data Transfer Object)	계층 간 데이터 전송을 목적으로 만든 객체. 엔티티의 공개 범위를 가리고 API 스펙을 독립적으로 유지

같은 이름 충돌 방지용 고유 번호	UUID	128비트 길이의 보편 고유 식별자 (RFC 9562). 충돌 가능성이 실무적으로 0으로 간주되는 무작위 문자열
URL을 폴더로 연결하기	정적 리소스 매핑	특정 URL 패턴을 파일 시스템의 디렉토리에 연결해 파일을 그대로 응답하게 하는 Spring MVC 설정 (`WebMvcConfigurer#addResourceHandlers`
주소로 사진 확인	정적 리소스 서빙	동적 렌더링 없이 지정된 파일을 HTTP 응답으로 그대로 내려주는 기능
그 이름으로 이어지는 DB의 방 번호	기본 키 (Primary Key)	한 테이블에서 각 행을 유일하게 식별하는 컬럼. `@Id`로 지정

이것만은 기억하자

- **JSON은 글자만 담는다.** 바이너리 파일을 JSON에 태우려면 글자로 변환해야 하고, Base64가 이 변환의 표준이다
- **Base64는 간단한 대신 약 33퍼센트 더 커진다.** 원본 3바이트가 글자 4개로 바뀌면서 크기·네트워크·메모리 비용이 한 번씩 더 붙는다
- **사진은 파일시스템, 메타데이터는 DB.** 바이너리를 DB에 직접 박지 않고 경로·UUID·생성 시각만 기록한다. 조회는 URL 한 줄로 끝난다
- `/uploads/**` **정적 리소스 매핑 한 줄로 서버가 그 폴더를 그대로 공개 URL로 만든다.** 파일을 내려주는 별도 API를 짜지 않아도 된다
- **10MB를 넘는 순간 이 방식은 무너진다.** 요청 바디 비대·힙 점유·DB 비대·업로드 한도가 한꺼번에 당겨 온다
- **다음 챕터에서는** 서버가 직접 받지 않고 사용자가 S3에 바로 올리도록 **Presigned URL**을 만듭니다. 업로드 완료 시점에는 **Lambda**가 썸네일을 만들고, Spring은 그 결과만 메타로 받아 적습니다 (Presigned URL + S3 + Lambda)